PV Site Layout & Electrical Loss Optimization — Task Brief

0. What this is, and what it isn't

This is **not** a `Component` in the `plugins/_template/` sense — it never `contribute()`s ports/flows/variables to the MILP, and it isn't a dispatchable technology. It's a **standalone geometric + electrical design tool**: given a site and a target DC capacity, compute a real panel layout, place inverters/transformers to minimize cable losses, and report the result (numbers + a picture). It runs *before* or *alongside* OptimaRE, not inside its solve. If it proves out, its OUTPUT (a set of loss percentages, an area-utilization figure) is what gets fed into OptimaRE later, as parameters — not this tool's own code.

So: build this as a plain, independent Python package. Don't use the `Component` / `contribution.py` / `physics.py` shape from `_template/` — it doesn't fit, and forcing it will waste your time.

1. The problem, precisely

Given:

- A site boundary (start with a plain rectangle: width × height in meters; a general polygon is a stretch goal, not day-one).
- A site location (latitude, longitude) — needed for row-spacing math.
- A target DC capacity to place (kWp) — this is the number OptimaRE's own optimizer would have already solved for (`rated_power × n_units` for a real project); you're being handed a target, not inventing one.
- A PV module spec: physical dimensions (length × width, meters) and rated power (Wp).
- Racking assumptions: fixed-tilt, tilt angle, row orientation.
- An inverter spec: rated AC power (kW).
- A transformer spec: rated capacity (kVA), no-load loss (kW), and load loss at rated capacity (kW) — these three numbers are always on a real transformer nameplate/datasheet; don't invent a formula that doesn't need them.
- A cable spec: conductor material (copper or aluminum) and cross-sectional area (mm^2) — or, to start simpler, just a direct "resistance per km" number so you don't need a wire-gauge lookup table for v1.

Compute and report:

1. How many modules fit, laid out in rows, respecting a real inter-row shading constraint (§3) — and whether the target DC capacity actually fits in the given site (report both: does it fit, and if not, how much DOES fit / how much land would be needed).
2. Real (x, y) coordinates for every module (or module row/string — see §4, you don't need one dot per physical panel).
3. How many inverters and transformers are needed, and their own (x, y) placement, chosen to minimize total DC and AC cable length — this is the actual optimization problem in this task (§4).
4. Cable losses (DC side: module strings → inverters; AC side: inverters → transformer) as a % of total energy, from real cable resistance and current, not a guessed number.
5. Transformer loss (%) from the standard two-term nameplate model (§5) — not a guessed number either.
6. A rendered picture: the site boundary, every row of panels, inverter and transformer markers, and the cable runs between them.

2. Explicitly out of scope for this pass

Don't build these — they're real gaps, but not this task, and trying to cover them now will make the core problem take too long to finish:

- Terrain slope/elevation (flat site only).
- Shading from anything other than PV rows themselves (no trees, buildings, terrain).
- Tracking racking (fixed-tilt only).
- Real trench/road-crossing-aware cable routing (straight-line/ Manhattan distance between points is fine).
- Cost (this task reports losses and land use, not ¤).
- Multiple MPPT strings per inverter, DC/AC ratio optimization, central vs. string inverter tradeoffs — assume one simple inverter type and size for the whole site.

3. Row spacing — this is a real, textbook formula, don't guess it

Panels in one row will shade the row behind them at low sun angles unless rows are spaced far enough apart. The standard approach: compute the **minimum sun elevation angle you're willing to tolerate full shading below** (a real design choice — e.g. "no inter-row shading for ±4 hours around solar noon on the winter solstice" is a common, citable convention), then set row pitch so no shadow falls on the row behind at that angle.

You need real solar-position equations for this (solar declination, hour angle, elevation angle as a function of latitude/day/time) — **do not derive these from scratch or let an AI tool invent them.** Use a real, standard reference:

- Duffie & Beckman, *Solar Engineering of Thermal Processes* — the standard solar-geometry textbook cited everywhere in this field, or
- NREL's Solar Position Algorithm (SPA) documentation, or
- `pvlib` (open-source Python, BSD-licensed — permissive, matches this project's own license policy, CLAUDE.md §1) already implements this exact geometry — you're welcome to use `pvlib.solarposition` directly rather than re-deriving it, IF you cite that this is what you're doing and understand what it's computing (don't call a function you can't explain).

Row pitch formula (once you have the critical elevation angle β):

```
vertical_rise = module_length * cos(tilt_angle)
pitch = vertical_rise / tan(β)
```

Ground coverage ratio (GCR) = module_length / pitch is a useful sanity number to report — real utility-scale sites run GCR ≈ 0.35–0.45; if your numbers come out wildly outside that range, something's wrong.

4. Inverter/transformer placement — a real facility-location problem

This is the actual optimization content of the task. Framed properly, it's a well-known problem class — look this up, don't invent an ad hoc heuristic from nothing:

1. Group modules into strings/blocks sized to roughly fill one inverter's rated capacity (block capacity ≈ inverter rated power). Each block has a real (x, y) centroid.
2. You now have a **k-median / weighted facility-location problem**: place `k` inverters (k = ceil(total DC kWp / inverter rated kW)) to minimize the sum of (block → assigned inverter) distances. Weighted **k-means clustering** on the block centroids is a good, standard, tractable starting algorithm — it won't be perfectly optimal, but it's a real, defensible, well-documented method (search "k-means for facility location" / "Weber problem"), not a guess.
3. Repeat one level up for the transformer(s): cluster the resulting inverter locations the same way to place the transformer (for a single-transformer site, this can just be the capacity-weighted centroid of all inverter locations).
4. Cable runs are then straight lines: each block → its assigned inverter (DC cabling), each inverter → its assigned transformer (AC cabling). Sum the lengths.

Stretch goal, only after the above works end to end: formulate this as a small MILP instead of k-means (minimize total weighted distance subject to each block assigned to exactly one inverter, each inverter within its rated capacity) and solve it with `highspy` (already an OptimaRE dependency, MIT-licensed) for a handful of candidate inverter sites. Don't start here — get the clustering version working and correct first.

5. Loss formulas — also real, also citable, don't invent these

Cable (I²R) loss, DC and AC segments separately:

```
R = resistivity * length / cross_sectional_area   # Ω
I = P / V                                         # DC, or
I = P / (V * sqrt(3) * power_factor)              # 3-phase AC
P_loss = I^2 * R
```

Copper resistivity ≈ 1.68×10^{-8} Ω·m, aluminum ≈ 2.82×10^{-8} Ω·m — real, checkable physical constants (any electrical engineering reference). Report loss as a % of the power flowing through that segment, and as an aggregate % of total site output.

Transformer loss, the standard two-term nameplate model:

```
P_loss = P_no_load + P_load_rated * (S / S_rated)^2
```

`P_no_load` (core/iron loss, constant regardless of loading) and `P_load_rated` (copper/winding loss AT rated load) come straight off a real transformer datasheet — this is IEC 60076's own standard model, not something to derive. `S` is the actual power flowing through it at the operating point you're evaluating.

6. Deliverables

A plain Python package (own `pyproject.toml`, so it's independently installable/testable — see `plugins/_template/pyproject.toml` for the shape, minus anything Component-specific), with roughly:

```
plugins/pv-layout-design/
  pyproject.toml
  README.md              # what you built, how to run it, real citations
  pv_layout/
    __init__.py
    geometry.py          # row spacing, panel placement, GCR
    placement.py         # inverter/transformer facility-location
    losses.py            # cable + transformer loss calculations
    plot.py              # renders the layout (matplotlib is fine)
  tests/
    test_geometry.py
    test_placement.py
    test_losses.py
  examples/
    run_example.py       # one real, worked example end to end
```

Every module/function needs a real docstring naming where its formula comes from (a textbook, a standard, a real library) — the same discipline the rest of this codebase already holds itself to (CLAUDE.md §6/§7: cite the source, disclose assumptions, never invent an equation). If you use an

AI tool to help write code, that discipline matters MORE, not less — an AI-suggested formula that looks plausible but isn't checkable against a real source doesn't clear this bar, and loss numbers off by even a little are exactly what your bid-side audience already told us they'll catch.

7. Acceptance / how we'll evaluate it

1. **Row spacing is correct**: hand-check one worked example against the row-pitch formula in §3 for a real latitude (pick your own site) — does the reported GCR land in the realistic ~0.35–0.45 range?
2. **The target DC capacity either fits or the tool honestly says it doesn't** — never silently place fewer panels than asked without saying so.
3. **Inverter/transformer placement genuinely reduces cable length** versus a naive baseline (e.g. "one inverter always at the corner of the site") — report both numbers so the improvement is visible, not just claimed.
4. **Loss numbers are traceable**: given the reported cable length and the input cable spec, I should be able to hand-recompute the loss % myself from §5's formula and get the same answer.
5. **The picture is real and readable**: site boundary, panel rows, inverter/transformer markers, and cable runs, all visibly distinct, for the same worked example as #1.
6. **Tests exist and pass** for each of geometry/placement/losses, using the SAME worked example as #1 as the fixture — so "does this still work" is a real, one-command check (`pytest plugins/pv-layout-design/tests/`), not something we re-verify by eye each time.

8. Suggested order of work

Don't try to build all of this at once. In order, each stage should be independently runnable and checkable before moving to the next:

1. Row spacing + panel placement on a rectangle for one fixed site/ module/latitude (§3). Get a plausible GCR before doing anything else.
2. Block the panels into inverter-sized groups; place inverters via k-means (§4, step 1-2); plot it. Look at the picture — does it look sane?
3. Add the transformer placement level (§4, step 3).
4. Add cable length calculation for the DC and AC runs from the layout you already have.
5. Add the loss formulas (§5) on top of those lengths.
6. Write the tests and the worked example last, against whatever you've already built and eyeballed as correct.

9. Where this goes next (context only — not part of this task)

If this works, its real integration point into OptimaRE is as a set of **derived parameters** — a computed cable-loss %, transformer-loss %, and land-area-used figure — feeding into the PV component's own (currently single lumped 0.8 `derating_factor`, see `core/optimare/components/pv/physics.py`) loss model as itemized inputs instead. That itemization is separate, later work, not something this task needs to wire up itself.